

GUIA DE ARQUITETURA DE SOFTWARE

Evolução de Aplicação Bancária POO para Arquitetura Web Modular

Este documento detalha o processo de transformação de um sistema bancário baseado em terminal (CLI) para uma arquitetura moderna orientada a serviços. A nova estrutura separa as responsabilidades em persistência de dados, lógica de negócios (API RESTful) e interface de usuário, garantindo escalabilidade e facilidade de manutenção.

Fase 1: Modelagem e Banco de Dados (Supabase / PostgreSQL)

A primeira etapa exige abandonar o armazenamento em memória (variáveis) ou arquivos locais em favor de um banco de dados relacional. O projeto utilizará o **Supabase**, que fornece um banco de dados PostgreSQL escalável na nuvem.

1.1 Modelagem Entidade-Relacionamento

Utilize um software de modelagem (como o **brModelo**) para desenhar o diagrama relacional. As três tabelas fundamentais são:

Tabela	Colunas Principais	Propósito
clientes	id (PK), nome, cpf (UNIQUE), data_cadastro	Armazenar os dados cadastrais dos titulares.
contas	id (PK), cliente_id (FK), agencia, numero, saldo	Gerenciar as contas vinculadas aos clientes e seus saldos atuais.
transacoes	id (PK), conta_id (FK), tipo (saque/ deposito), valor, data_hora	Garantir o histórico e a rastreabilidade das operações.

1.2 Setup no Supabase

- Crie um projeto no painel do Supabase.
- Acesse o SQL Editor e execute as *queries* de criação de tabela (DDL).
- Em *Project Settings > API*, copie a URL e a anon key. Esses dados conectarão seu backend em Python ao banco de dados.

Fase 2: Construção da API REST (Python + FastAPI)

A lógica do sistema, onde residem os princípios de Programação Orientada a Objetos (POO), passará a operar como um servidor web através do **FastAPI**.

Por que FastAPI? É rápido, possui validação de dados automática via Pydantic e gera documentação da API automaticamente (Swagger).

2.1 Estrutura de Diretórios do Backend

```
backend/
├── main.py           # Instância do app FastAPI e roteamento principal
├── database.py       # Conexão com o Supabase (supabase-py)
├── models.py         # Classes Pydantic para validação de dados
├── crud.py           # Lógica de inserção e busca no banco
├── core/
│   └── conta_bancaria.py # Sua lógica POO original (validação de regras)
└── requirements.txt  # fastapi, uvicorn, supabase, pydantic
```

2.2 Exemplo de Integração (Endpoint de Depósito)

Em `main.py`, as funções não usam mais `input()` ou `print()`. Elas recebem requisições HTTP e devolvem respostas JSON:

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from database import supabase

app = FastAPI()

class TransacaoIn(BaseModel):
    conta_id: int
    valor: float

@app.post("/api/contas/deposito")
def realizar_deposito(transacao: TransacaoIn):
    if transacao.valor <= 0:
        raise HTTPException(status_code=400, detail="Valor inválido")

    # 1. Registrar a transação no banco (Supabase)
    supabase.table('transacoes').insert({...}).execute()

    # 2. Atualizar o saldo da conta
    # ...lógica de update...

    return {"status": "sucesso", "mensagem": "Depósito realizado"}
```

Fase 3: Interface de Usuário (React)

O **React** será responsável pela construção visual. Ele rodará no navegador do usuário e fará requisições (chamadas HTTP) para a API FastAPI.

3.1 Modularização dos Componentes

A interface deve ser dividida em componentes reutilizáveis. Uma estrutura sugerida:

- `App.jsx`: Gerencia o estado global e a navegação.
- `Header.jsx`: Cabeçalho com o nome do cliente e agência/conta.
- `DashboardPanel.jsx`: Exibe o saldo atual em destaque.
- `TransactionForm.jsx`: Formulário com botões de "Depositar", "Sacar" e "Transferir", validando os inputs antes de enviar para a API.
- `StatementList.jsx`: Tabela ou lista que consome e exibe o histórico de transações.

3.2 Consumindo a API Python

No componente de formulário (`TransactionForm.jsx`), o React aciona o backend:

```
const realizarDeposito = async (valor) => {
  try {
    const response = await fetch("http://localhost:8000/api/contas/deposito", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ conta_id: 1, valor: parseFloat(valor) })
    });
    const data = await response.json();
    alert(data.mensagem); // Atualiza a UI após sucesso
  } catch (error) {
    console.error("Erro na transação", error);
  }
};
```

Fase 4: Empacotamento e Implantação (Docker)

Para garantir que o ambiente de desenvolvimento seja idêntico ao de produção, o sistema será "containerizado" usando **Docker**. O projeto deve conter um arquivo `docker-compose.yml` na raiz, subindo o back e o front de uma só vez.

4.1 Arquitetura do Docker Compose

```
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - SUPABASE_URL=sua_url_aqui
      - SUPABASE_KEY=sua_key_aqui

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
```

Com apenas o comando `docker-compose up -d`, todo o ecossistema bancário (API em Python e Interface em React) será iniciado simultaneamente, totalmente isolado e configurado.